

Linear Bounding Volume Hierarchy (LBVH): Build and Traversal Implementation Document

Zixuan Vicky Lu

December 4, 2025

1 LBVH Data Structures

1.1 Axis-Aligned Bounding Box (AABB)

$$\text{AABB} = \{\mathbf{min}, \mathbf{max}\} \in \mathbb{R}^3 \times \mathbb{R}^3 \quad (1)$$

Two AABBs A and B intersect if and only if:

$$A.\text{max}_i \geq B.\text{min}_i \quad \wedge \quad A.\text{min}_i \leq B.\text{max}_i, \quad \forall i \in \{x, y, z\} \quad (2)$$

1.2 BVH Node Structure

```
struct BVHNode {
    AABB    bounds[2];      // Left and right child bounds
    uint32  leftIdx;        // Left child index (MSB=1 if leaf)
    uint32  rightIdx;       // Right child index (MSB=1 if leaf)
    uint32  parentIdx;      // Parent node index
    uint32  fence;          // Range boundary for self-collision culling
};
```

2 LBVH Construction

2.1 Step 1: Compute Morton Codes

Morton codes map 3D coordinates to a 1D space-filling curve that preserves spatial locality.

Algorithm 1 Compute Morton Code

```
1: function MORTON3D( $x, y, z$ ) ▷ Each coordinate in  $[0, 1023]$ 
2:    $x \leftarrow \text{EXPANDBITS}(x)$  ▷ Interleave bits
3:    $y \leftarrow \text{EXPANDBITS}(y)$ 
4:    $z \leftarrow \text{EXPANDBITS}(z)$ 
5:   return  $(x \ll 2) \vee (y \ll 1) \vee z$ 
6: end function
```

Algorithm 2 Expand Bits for Morton Code

```
1: function EXPANDBITS( $v$ ) ▷ Insert two 0-bits between each bit
2:    $v \leftarrow (v \cdot 0x00010001) \wedge 0xFF0000FF$ 
3:    $v \leftarrow (v \cdot 0x00000101) \wedge 0x0F00F00F$ 
4:    $v \leftarrow (v \cdot 0x00000011) \wedge 0xC30C30C3$ 
5:    $v \leftarrow (v \cdot 0x00000005) \wedge 0x49249249$ 
6:   return  $v$ 
7: end function
```

2.2 Step 2: Sort by Morton Code

Perform a radix sort on $(\text{mortonCode}, \text{objectID})$ pairs:

$$\text{sorted} \leftarrow \text{RadixSort}(\{(\text{morton}_i, i) : i \in [0, N)\}) \quad (3)$$

2.3 Step 3: Build Internal Nodes (Karras 2012)

Using the sorted Morton codes, we construct the binary radix tree in parallel.

Algorithm 3 Determine Node Range

```
1: function DETERMINERANGE( $\text{mortonCodes}, N, \text{idx}$ )
2:   if  $\text{idx} = 0$  then
3:     return  $(0, N - 1)$ 
4:   end if
5:    $\text{selfCode} \leftarrow \text{MergeIdx}(\text{mortonCodes}[\text{idx}], \text{idx})$ 
6:    $\delta_L \leftarrow \text{CLZ}(\text{selfCode} \oplus \text{MergeIdx}(\text{mortonCodes}[\text{idx} - 1], \text{idx} - 1))$ 
7:    $\delta_R \leftarrow \text{CLZ}(\text{selfCode} \oplus \text{MergeIdx}(\text{mortonCodes}[\text{idx} + 1], \text{idx} + 1))$ 
8:    $d \leftarrow \text{sign}(\delta_R - \delta_L)$  ▷ Direction of range
9:    $\delta_{\min} \leftarrow \min(\delta_L, \delta_R)$ 
10: ▷ Binary search for range length
11:    $l_{\max} \leftarrow 2$ 
12:   while  $0 \leq \text{idx} + d \cdot l_{\max} < N$  and  $\delta(\text{idx}, \text{idx} + d \cdot l_{\max}) > \delta_{\min}$  do
13:      $l_{\max} \leftarrow l_{\max} \cdot 2$ 
14:   end while
15:    $l \leftarrow 0, \quad t \leftarrow l_{\max}/2$ 
16:   while  $t > 0$  do
17:     if  $\delta(\text{idx}, \text{idx} + (l + t) \cdot d) > \delta_{\min}$  then
18:        $l \leftarrow l + t$ 
19:     end if
20:      $t \leftarrow t/2$ 
21:   end while
22:   return  $(\min(\text{idx}, \text{idx} + l \cdot d), \max(\text{idx}, \text{idx} + l \cdot d))$ 
23: end function
```

Algorithm 4 Find Split Position

```
1: function FINDSPLIT(mortonCodes, first, last)
2:    $\delta_{\text{node}} \leftarrow \text{CLZ}(\text{MergeIdx}(\text{first}) \oplus \text{MergeIdx}(\text{last}))$ 
3:   split  $\leftarrow$  first
4:   stride  $\leftarrow$  last - first
5:   repeat
6:     stride  $\leftarrow \lceil \text{stride}/2 \rceil$ 
7:     middle  $\leftarrow$  split + stride
8:     if middle < last and  $\delta(\text{first}, \text{middle}) > \delta_{\text{node}}$  then
9:       split  $\leftarrow$  middle
10:    end if
11:  until stride  $\leq 1$ 
12:  return split
13: end function
```

2.4 Step 4: Refit AABBs (Bottom-Up)

Update internal node bounds by traversing from leaves to root using atomic flags.

Algorithm 5 Refit AABB (Per Leaf Thread)

```
1: function REFITAABB(nodes, leafParents, aabbs, objIDs, flags, leafIdx)
2:   bound  $\leftarrow$  aabbs[objIDs[leafIdx]]
3:   parent  $\leftarrow$  leafParents[leafIdx]
4:   while true do
5:     isRight  $\leftarrow$  parent  $\wedge$  0x80000000
6:     parent  $\leftarrow$  parent  $\wedge$  0x7FFFFFFF
7:     nodes[parent].bounds[isRight]  $\leftarrow$  bound
8:     if ATOMICEXCHANGE(flags[parent], 1) = 0 then
9:       return ▷ First thread to arrive, wait for sibling
10:    end if
11:    THREADFENCE ▷ Ensure memory coherency
12:    if parent = 0 then
13:      return ▷ Reached root
14:    end if
15:    bound  $\leftarrow$  bound  $\cup$  nodes[parent].bounds[1 - isRight]
16:    parent  $\leftarrow$  nodes[parent].parentIdx
17:  end while
18: end function
```

3 Two-Pass Traversal Algorithm

The two-pass approach avoids the need for a large pre-allocated output buffer by first counting collisions, then allocating exactly the required memory.

3.1 Overview

1. **Pass 1 (Count):** For each query AABB, count the number of collisions
2. **Build Offsets:** Compute exclusive prefix sum of counts to get write offsets
3. **Pass 2 (Fill):** Traverse again and write collision pairs to the buffer

3.2 Pass 1: Counting Collisions

Algorithm 6 Two-Pass Traversal: Count Phase

```

1: procedure COUNTCOLLISIONS(nodes, objIDs, queryAABBs, counts,  $N$ )
2:   for  $q \leftarrow 0$  to  $N - 1$  in parallel do                                 $\triangleright$  One thread per query
3:     queryAABB  $\leftarrow$  queryAABBs[ $q$ ]
4:     count  $\leftarrow 0$ 
5:     stack[0]  $\leftarrow 0$                                                  $\triangleright$  Push root node
6:     stackPtr  $\leftarrow 1$ 
7:     while stackPtr  $> 0$  do
8:       stackPtr  $\leftarrow$  stackPtr  $- 1$ 
9:       nodeId  $\leftarrow$  stack[stackPtr]                                     $\triangleright$  Pop
10:      isLeaf  $\leftarrow$  nodeId  $\wedge$  0x80000000
11:      nodeId  $\leftarrow$  nodeId  $\wedge$  0x7FFFFFFF
12:      if isLeaf then
13:        if nodeId  $> q$  then                                             $\triangleright$  Avoid duplicates for self-query
14:          count  $\leftarrow$  count  $+ 1$ 
15:        end if
16:      else
17:        node  $\leftarrow$  nodes[nodeId]
18:        if max(nodeId, node.fence)  $> q$  then                             $\triangleright$  Subtree culling
19:          if node.bounds[0]  $\cap$  queryAABB  $\neq \emptyset$  then
20:            stack[stackPtr]  $\leftarrow$  node.leftIdx                         $\triangleright$  Push left
21:            stackPtr  $\leftarrow$  stackPtr  $+ 1$ 
22:          end if
23:          if node.bounds[1]  $\cap$  queryAABB  $\neq \emptyset$  then
24:            stack[stackPtr]  $\leftarrow$  node.rightIdx                         $\triangleright$  Push right
25:            stackPtr  $\leftarrow$  stackPtr  $+ 1$ 
26:          end if
27:        end if
28:      end if
29:    end while
30:    counts[ $q$ ]  $\leftarrow$  count
31:  end for
32: end procedure

```

3.3 Build Offsets (Exclusive Prefix Sum)

After counting, compute write offsets using an exclusive prefix sum:

$$\text{offsets}[0] = 0 \tag{4}$$

$$\text{offsets}[i] = \sum_{j=0}^{i-1} \text{counts}[j], \quad i \in [1, N] \tag{5}$$

The total number of collisions is:

$$\text{totalCollisions} = \text{offsets}[N] = \sum_{i=0}^{N-1} \text{counts}[i] \tag{6}$$

Algorithm 7 Build Offsets

```
1: procedure BUILD_OFFSETS(counts, offsets,  $N$ )
2:   offsets  $\leftarrow$  EXCLUSIVE_PREFIX_SUM(counts)
3:   totalCollisions  $\leftarrow$  offsets[ $N - 1$ ] + counts[ $N - 1$ ]
4:   ALLOCATE(collisionBuffer, totalCollisions)
5: end procedure
```

3.4 Pass 2: Filling Collision Buffer

Algorithm 8 Two-Pass Traversal: Fill Phase

```
1: procedure FILL_COLLISIONS(nodes, objIDs, queryAABBs, offsets, buffer,  $N$ )
2:   for  $q \leftarrow 0$  to  $N - 1$  in parallel do ▷ One thread per query
3:     queryAABB  $\leftarrow$  queryAABBs[ $q$ ]
4:     writeIdx  $\leftarrow$  offsets[ $q$ ] ▷ Start writing at offset
5:     stack[0]  $\leftarrow$  0 ▷ Push root node
6:     stackPtr  $\leftarrow$  1
7:     while stackPtr > 0 do
8:       stackPtr  $\leftarrow$  stackPtr - 1
9:       nodeIdx  $\leftarrow$  stack[stackPtr] ▷ Pop
10:      isLeaf  $\leftarrow$  nodeIdx  $\wedge$  0x80000000
11:      nodeIdx  $\leftarrow$  nodeIdx  $\wedge$  0x7FFFFFFF
12:      if isLeaf then
13:        if nodeIdx >  $q$  then ▷ Avoid duplicates
14:          buffer[writeIdx]  $\leftarrow$  (objIDs[ $q$ ], objIDs[nodeIdx])
15:          writeIdx  $\leftarrow$  writeIdx + 1
16:        end if
17:      else
18:        node  $\leftarrow$  nodes[nodeIdx]
19:        if max(nodeIdx, node.fence) >  $q$  then ▷ Subtree culling
20:          if node.bounds[0]  $\cap$  queryAABB  $\neq \emptyset$  then
21:            stack[stackPtr]  $\leftarrow$  node.leftIdx ▷ Push left
22:            stackPtr  $\leftarrow$  stackPtr + 1
23:          end if
24:          if node.bounds[1]  $\cap$  queryAABB  $\neq \emptyset$  then
25:            stack[stackPtr]  $\leftarrow$  node.rightIdx ▷ Push right
26:            stackPtr  $\leftarrow$  stackPtr + 1
27:          end if
28:        end if
29:      end if
30:    end while
31:  end for
32: end procedure
```

4 Complete Two-Pass Pipeline

Algorithm 9 Complete Two-Pass BVH Query

```

1: procedure TWOPASSQUERY(bvh, queryAABBs,  $N$ )
2:                                     ▷ Pass 1: Count collisions
3:   counts  $\leftarrow$  array of size  $N$ 
4:   COUNTCOLLISIONS(bvh.nodes, bvh.objIDs, queryAABBs, counts,  $N$ )
5:
6:                                     ▷ Build offsets with exclusive prefix sum
7:   offsets  $\leftarrow$  EXCLUSIVEPREFIXSUM(counts)
8:   totalCollisions  $\leftarrow$  offsets[ $N - 1$ ] + counts[ $N - 1$ ]
9:
10:                                     ▷ Allocate exact buffer size
11:   buffer  $\leftarrow$  allocate(totalCollisions  $\times$  sizeof(int2))
12:
13:                                     ▷ Pass 2: Fill collision pairs
14:   FILLCOLLISIONS(bvh.nodes, bvh.objIDs, queryAABBs, offsets, buffer,  $N$ )
15:   return (buffer, totalCollisions)
16: end procedure

```

5 Single-Pass traverse algorithm

The single-pass traversal algorithm performs collision detection in a single kernel launch by utilising a two-level buffering strategy to reduce duplicate visits to global memory. Each thread independently traverses the BVH using a local stack, while collision results are collaboratively flushed to global memory through shared memory buffering.

To achieve a single-pass traverse, we have to face atomic convention while maintaining a global counter point to the back of the collision buffer. The single-pass traversal algorithm performs collision detection in a single kernel launch by utilizing a two-level buffering strategy to reduce atomic contention on global memory. Each thread independently traverses the BVH using a local stack, while collision results are collaboratively flushed to global memory through shared memory buffering. When the shared buffer is full, a thread that has pending collisions re-pushes the current node onto its stack and exits the inner traversal loop. After the buffer is flushed, threads resume traversal from their saved state. This mechanism ensures no collisions are lost while maintaining synchronisation constraints.

The algorithm terminates when either (1) all threads have completed their traversals with a partially-filled buffer, or (2) the global output buffer reaches capacity.

Algorithm 10 Single-Pass BVH Query with Shared Memory Buffering

```
1: procedure SINGLEPASSQUERY(result, counter, maxRes, nodes, objIDs, queryAABBs,  $N$ )
2:   tid  $\leftarrow$  globalThreadId
3:   queryAABB  $\leftarrow$  queryAABBs[objIDs[tid]]
4:
5:   shared buffer[BUFFER_SIZE], bufferCount  $\leftarrow$  0, globalWriteIdx
6:   local stack[STACK_SIZE], stackPtr  $\leftarrow$  1, stack[0]  $\leftarrow$  root
7:
8:   while true do
9:     SYNCTHREADS
10:    while stackPtr > 0 and tid <  $N$  do
11:      (nodeIdx, isLeaf)  $\leftarrow$  Pop(stack)
12:      if isLeaf then
13:        if nodeIdx > tid then ▷ Avoid duplicate pairs
14:          slot  $\leftarrow$  ATOMICADD(bufferCount, 1)
15:          if slot  $\geq$  BUFFER_SIZE then
16:            Push(stack, nodeIdx, isLeaf) ▷ Re-push, wait for flush
17:            break
18:          end if
19:          buffer[slot]  $\leftarrow$  (objIDs[nodeIdx], objIDs[tid])
20:        end if
21:      else
22:        node  $\leftarrow$  nodes[nodeIdx]
23:        if max(nodeIdx, node.fence) > tid then ▷ Subtree pruning
24:          if node.leftBound  $\cap$  queryAABB  $\neq \emptyset$  then
25:            Push(stack, node.leftIdx)
26:          end if
27:          if node.rightBound  $\cap$  queryAABB  $\neq \emptyset$  then
28:            Push(stack, node.rightIdx)
29:          end if
30:        end if
31:      end if
32:    end while
33:
34:    ▷ Flush shared buffer to global memory
35:    SYNCTHREADS
36:    count  $\leftarrow$  min(bufferCount, BUFFER_SIZE)
37:    if threadIdx = 0 then
38:      globalWriteIdx  $\leftarrow$  ATOMICADD(counter, count)
39:    end if
40:    SYNCTHREADS
41:    if globalWriteIdx  $\geq$  maxRes or count = 0 then
42:      return
43:    end if
44:    COOPERATIVECOPY(result[globalWriteIdx :], buffer, count)
45:    bufferCount  $\leftarrow$  0
46:    if count < BUFFER_SIZE then
47:      return ▷ All threads finished
48:    end if
49:  end while
50: end procedure
```

6 The stackless algorithm

Traditional BVH traversal algorithms maintain an explicit stack to track nodes pending visitation during depth-first traversal. On GPUs, this per-thread stack consumes valuable register or local memory resources, limiting occupancy and reducing parallelism. For N primitives, a balanced BVH requires $O(\log N)$ stack depth—up to 32 entries for billions of objects. The stackless approach eliminates this overhead by encoding traversal state directly into the tree structure. Each node stores an escape pointer that indicates where to continue traversal when the current subtree is either fully explored or pruned due to a failed intersection test. This trades the runtime stack management for a modest increase in node size and a one-time preprocessing step during tree construction.

Escape Index: Where to Go When Subtree is Skipped

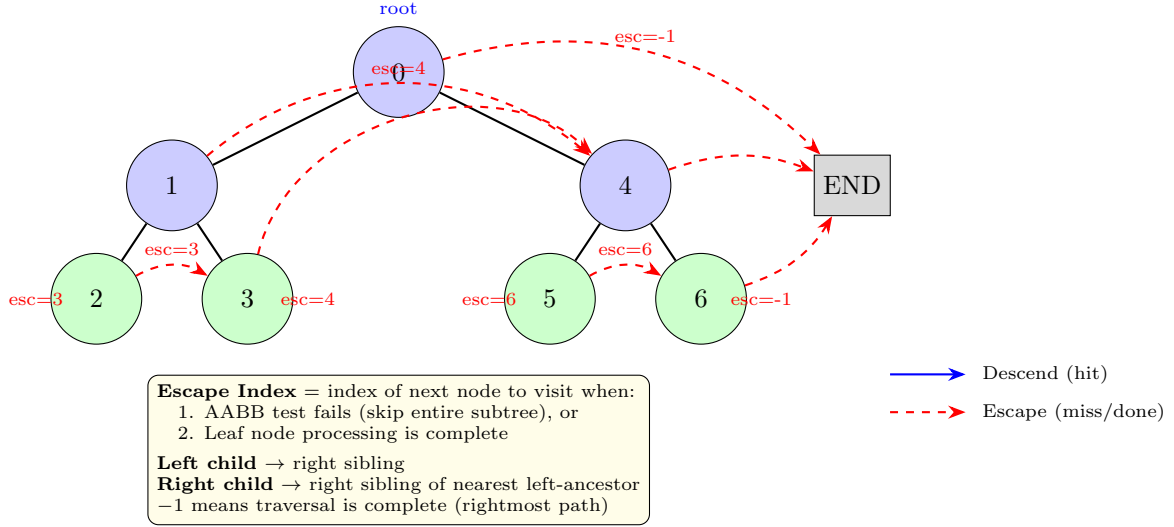
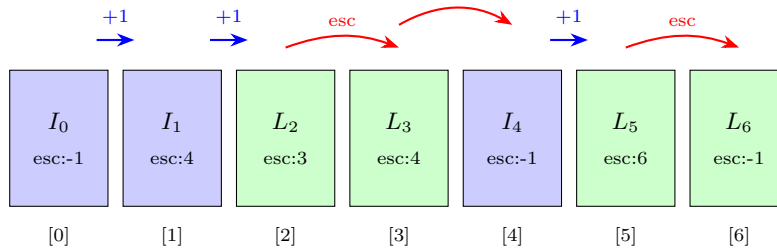


Figure 1: Escape index definition. Left children escape to right sibling; right children escape further up.

Stackless BVH: Pre-order Array Layout



Hit (descend): next = current + 1
Miss (escape): next = escape[current]

Figure 2: Pre-order linearization. Left child is always at index+1. Left children escape to right sibling.


```

struct __align__(32)
StacklessNode {
    Bound<float> bound;
    int data;    // -1 for internal, primitive index for leaf
    int escape; // Index of next node to visit on miss/finish
};

```

The total node size is 32 bytes, aligned for efficient GPU memory access. The pre-order layout ensures that when traversing into a child, the left child is always at index current + 1, eliminating the need to store explicit child pointers for the descending direction. The escape pointer encodes the rope of the node—the first node that should be visited after the current subtree is fully processed. For nodes on the rightmost path of the tree, the escape pointer is -1 , signalling completion of the traversal.

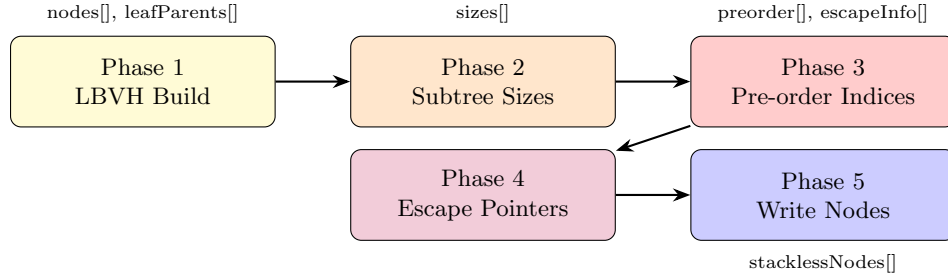


Figure 3: Stackless BVH build pipeline. Each phase computes data needed by subsequent phases.

Algorithm 11 Complete Stackless BVH Build Pipeline

```

1: procedure BUILDSTACKLESSBVH(aabbs,  $N$ )
2:                                      $\triangleright$  Phase 1: Standard LBVH Construction
3:   sceneBound  $\leftarrow$  COMPUTESCENEBOUND(aabbs,  $N$ )
4:   morton  $\leftarrow$  COMPUTEMORTONCODES(aabbs, sceneBound,  $N$ )
5:   (morton, objIDs)  $\leftarrow$  RADIXSORT(morton, [0,  $N$ ))
6:   nodes  $\leftarrow$  BUILDRADIXTREE(morton,  $N$ )
7:   REFITAABBS(nodes, aabbs, objIDs)
8:
9:                                      $\triangleright$  Phase 2: Subtree Size Calculation
10:  sizes  $\leftarrow$  CALCSUBTREESIZES(nodes, leafParents,  $N$ )
11:
12:                                      $\triangleright$  Phase 3: Pre-order Index Computation
13:  (preorder, escapeInfo)  $\leftarrow$  COMPUTEPREORDERINDICES(nodes, sizes)
14:  (leafPreorder, leafEscape)  $\leftarrow$  COMPUTELEAFPREORDER(nodes, sizes)
15:
16:                                      $\triangleright$  Phase 4 & 5: Write Linearized Nodes
17:  output  $\leftarrow$  allocate( $2N - 1$ )
18:  WRITESTACKLESSNODES(nodes, output, preorder, escapeInfo, ...)
19:  WRITESTACKLESSLEAVES(aabbs, objIDs, output, leafPreorder, ...)
20:
21:  return output
22: end procedure

```

Phase 2: Subtree Size Computation (Bottom-Up)

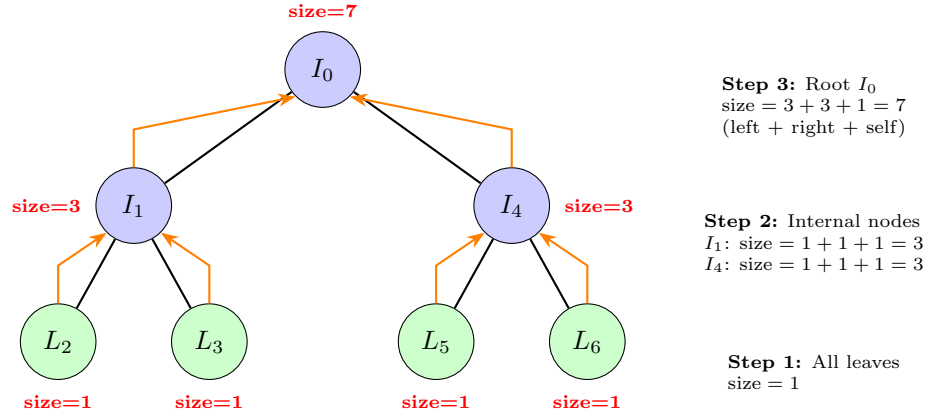


Figure 4: Subtree size computation. Sizes propagate bottom-up: $\text{size}(\text{node}) = \text{size}(\text{left}) + \text{size}(\text{right}) + 1$.

Atomic Synchronization: Two Children Meet at Parent

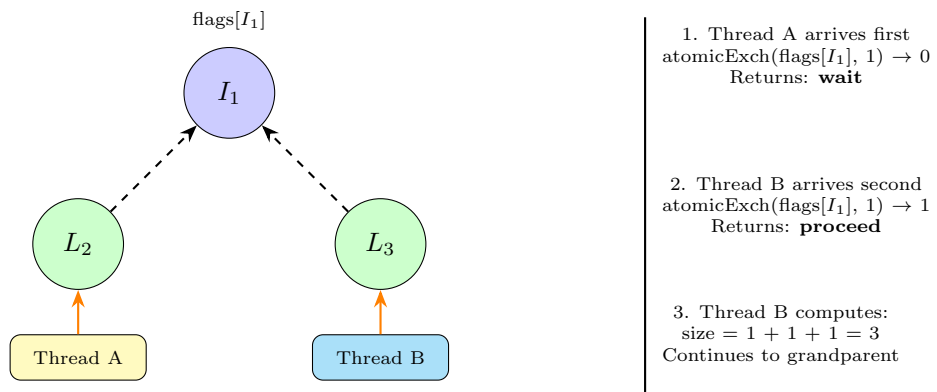


Figure 5: Atomic synchronization ensures both children report before parent size is computed. First thread waits; second thread computes and continues upward.

Final Subtree Sizes (used for Pre-order Computation)

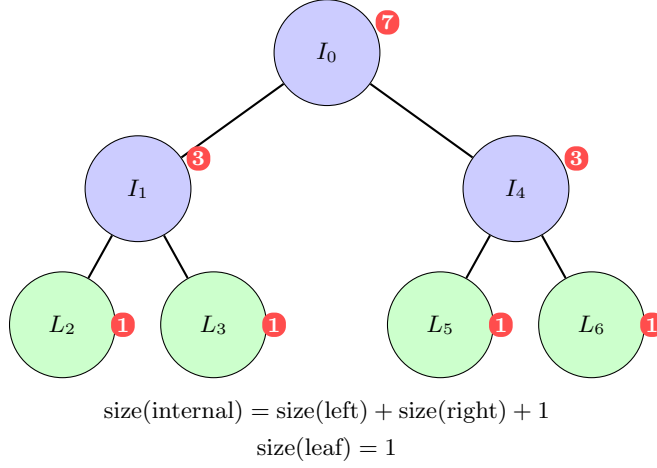


Figure 6: Each node's subtree size indicates how many nodes are in its subtree (including itself).

Algorithm 12 Phase 2: Subtree Size Calculation (Bottom-Up)

```

1: procedure CALCSUBTREESIZES(nodes, leafParents, flags, sizes,  $N$ )
2:   for leafIdx  $\leftarrow 0$  to  $N - 1$  in parallel do
3:     size  $\leftarrow 1$  ▷ Leaf size is 1
4:     parent  $\leftarrow$  leafParents[leafIdx]
5:     while parent  $\neq$  NULL do
6:       parentIdx  $\leftarrow$  parent  $\wedge$  0x7FFFFFFF
7:       siblingSize  $\leftarrow$  ATOMICEXCHANGE(flags[parentIdx], size)
8:       if siblingSize = 0 then
9:         return ▷ First child arrived, wait for sibling
10:      end if
11:      size  $\leftarrow$  size + siblingSize + 1 ▷ Both children + parent
12:      sizes[parentIdx]  $\leftarrow$  size
13:      THREADFENCE
14:      if parentIdx = 0 then
15:        return ▷ Reached root
16:      end if
17:      parent  $\leftarrow$  nodes[parentIdx].parentIdx
18:    end while
19:  end for
20: end procedure

```

Tree Structure with Pre-order Indices

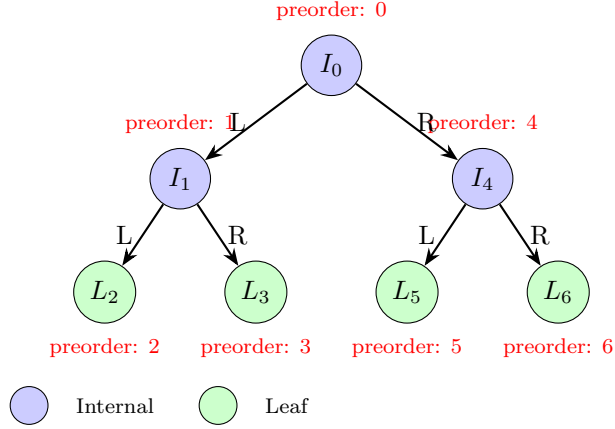


Figure 7: Binary tree with 3 internal nodes (I) and 4 leaves (L). Numbers indicate pre-order traversal indices.

Linearized Pre-order Array Layout

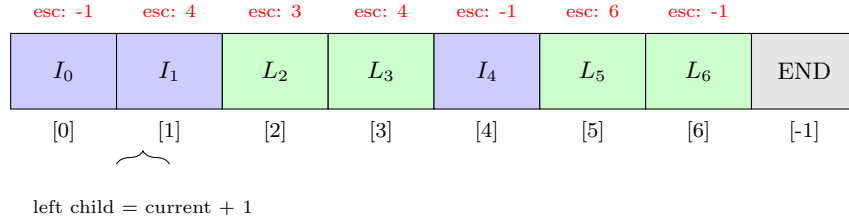


Figure 8: Nodes stored in pre-order. Left child is always at index+1. Escape pointers (red) indicate where to jump on miss.

Escape Pointers: Skip Subtrees on Miss

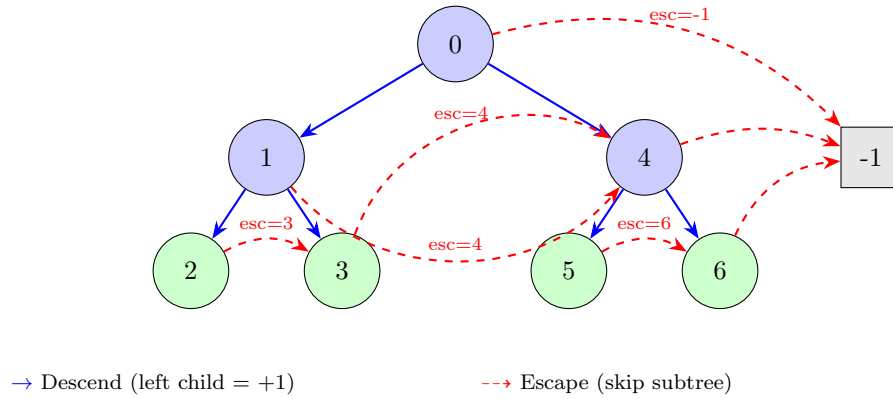
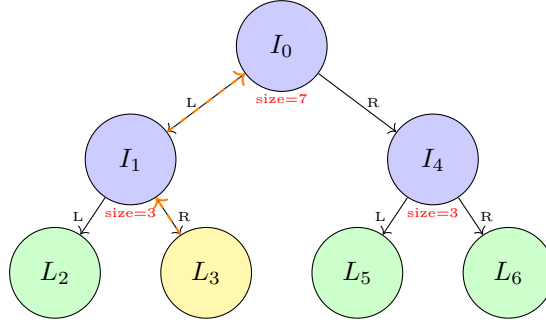


Figure 9: Escape pointers: left children escape to right sibling; right children escape to right sibling of nearest left-ancestor. Rightmost path escapes to -1.

Computing Pre-order Index: Walk to Root



Example: Compute preorder of L_3

Start at L_3 , walk to root:

1. L_3 is **right** child of I_1
 $\Rightarrow$ add $1 + \text{size}(L_2) = 1 + 1 = 2$
2. I_1 is **left** child of I_0
 $\Rightarrow$ add 1
3. I_0 is root, stop

Total: $2 + 1 = 3$ ✓

Figure 10: Pre-order index computation by walking to root. Right children add $1 + \text{size}(\text{leftSibling})$; left children add 1.

Algorithm 13 Phase 3: Pre-order Index Computation

```

1: function GETSUBTREESIZE(nodeIdx, sizes)
2:   if nodeIdx  $\wedge$  0x80000000 then                                      $\triangleright$  Is leaf
3:     return 1
4:   end if
5:   return sizes[nodeIdx]
6: end function
7:
8: procedure COMPUTEPREORDERINDICES(nodes, sizes, preorder, escapeInfo, N)
9:   for nodeIdx  $\leftarrow$  0 to  $N - 2$  in parallel do                        $\triangleright$  Internal nodes
10:    pos  $\leftarrow$  0
11:    escapeAncestor  $\leftarrow$  -1
12:    current  $\leftarrow$  nodeIdx
13:    while current  $\neq$  0 do                                            $\triangleright$  Walk to root
14:      parentInfo  $\leftarrow$  nodes[current].parentIdx
15:      isRight  $\leftarrow$  parentInfo  $\wedge$  0x80000000
16:      parentIdx  $\leftarrow$  parentInfo  $\wedge$  0x7FFFFFFF
17:      if isRight then
18:        leftIdx  $\leftarrow$  nodes[parentIdx].leftIdx
19:        pos  $\leftarrow$  pos + 1 + GETSUBTREESIZE(leftIdx, sizes)
20:      else
21:        pos  $\leftarrow$  pos + 1                                              $\triangleright$  Add 1 for parent
22:        if escapeAncestor = -1 then
23:          escapeAncestor  $\leftarrow$  parentIdx                              $\triangleright$  First left-child ancestor
24:        end if
25:      end if
26:      current  $\leftarrow$  parentIdx
27:    end while
28:    preorder[nodeIdx]  $\leftarrow$  pos
29:    escapeInfo[nodeIdx]  $\leftarrow$  escapeAncestor
30:  end for
31: end procedure

```

Algorithm 14 Phase 3b: Leaf Pre-order Index Computation

```
1: procedure COMPUTELEAFPREORDER(nodes, leafParents, sizes, leafPreorder, leafEscape,  $N$ )
2:   for leafIdx  $\leftarrow 0$  to  $N - 1$  in parallel do
3:     pos  $\leftarrow 0$ 
4:     escapeAncestor  $\leftarrow -1$ 
5:     parentInfo  $\leftarrow$  leafParents[leafIdx]
6:     isRight  $\leftarrow$  parentInfo  $\wedge$  0x80000000
7:     parentIdx  $\leftarrow$  parentInfo  $\wedge$  0x7FFFFFFF
8:     if isRight then
9:       pos  $\leftarrow$  pos + 1 + GETSUBTREESIZE(nodes[parentIdx].leftIdx, sizes)
10:    else
11:      pos  $\leftarrow$  pos + 1
12:      escapeAncestor  $\leftarrow$  parentIdx
13:    end if
14:    while parentIdx  $\neq 0$  do ▷ Continue to root
15:      gpInfo  $\leftarrow$  nodes[parentIdx].parentIdx
16:      gpIsRight  $\leftarrow$  gpInfo  $\wedge$  0x80000000
17:      gpIdx  $\leftarrow$  gpInfo  $\wedge$  0x7FFFFFFF
18:      if gpIsRight then
19:        pos  $\leftarrow$  pos + 1 + GETSUBTREESIZE(nodes[gpIdx].leftIdx, sizes)
20:      else
21:        pos  $\leftarrow$  pos + 1
22:        if escapeAncestor = -1 then
23:          escapeAncestor  $\leftarrow$  gpIdx
24:        end if
25:      end if
26:      parentIdx  $\leftarrow$  gpIdx
27:    end while
28:    leafPreorder[leafIdx]  $\leftarrow$  pos
29:    leafEscape[leafIdx]  $\leftarrow$  escapeAncestor
30:  end for
31: end procedure
```

Algorithm 15 Phase 4 & 5: Escape Resolution and Node Writing

```
1: function RESOLVEESCAPE(escapeAncestor, nodes, preorder, leafPreorder)
2:   if escapeAncestor < 0 then
3:     return -1 ▷ No escape, rightmost path
4:   end if
5:   rightIdx ← nodes[escapeAncestor].rightIdx
6:   if rightIdx ∧ 0x80000000 then ▷ Right sibling is leaf
7:     return leafPreorder[rightIdx ∧ 0x7FFFFFFF]
8:   else
9:     return preorder[rightIdx]
10:  end if
11: end function
12:
13: procedure WRITESTACKLESSNODES(nodes, output, preorder, escapeInfo, leafPreorder)
14:   for nodeIdIdx ← 0 to  $N_{\text{internal}} - 1$  in parallel do
15:     pos ← preorder[nodeIdx]
16:     escape ← RESOLVEESCAPE(escapeInfo[nodeIdx], ...)
17:     node ← nodes[nodeIdx]
18:     snode.bound ← node.bound[0] ∪ node.bound[1]
19:     snode.data ← -1 ▷ Internal node marker
20:     snode.escape ← escape
21:     output[pos] ← snode
22:   end for
23: end procedure
24:
25: procedure WRITESTACKLESSLEAVES(aabbs, objIDs, output, leafPreorder, leafEscape, ...)
26:   for leafIdx ← 0 to  $N - 1$  in parallel do
27:     pos ← leafPreorder[leafIdx]
28:     escape ← RESOLVEESCAPE(leafEscape[leafIdx], ...)
29:     objId ← objIDs[leafIdx]
30:     snode.bound ← aabbs[objId]
31:     snode.data ← objId ▷ Primitive index
32:     snode.escape ← escape
33:     output[pos] ← snode
34:   end for
35: end procedure
```

Stackless Traversal Algorithm

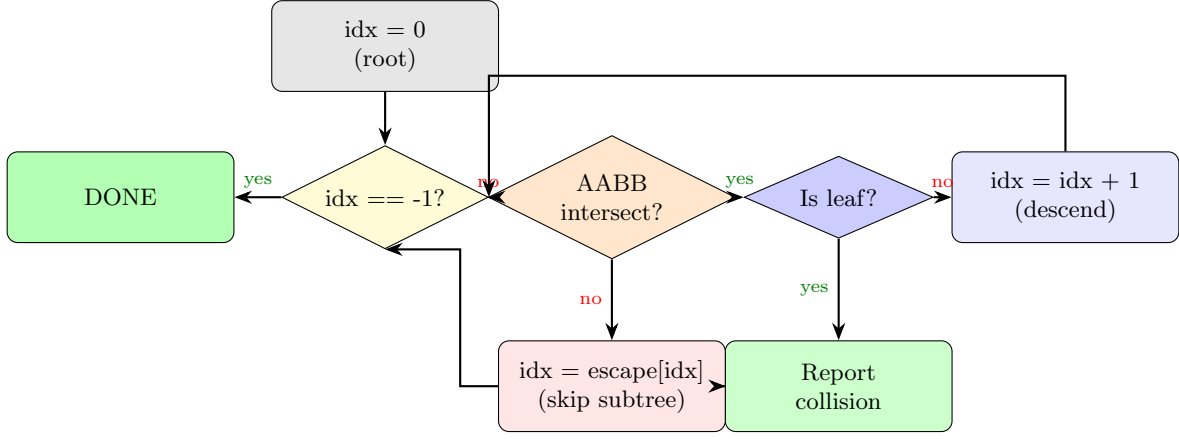


Figure 11: Stackless traversal flowchart. The escape pointer eliminates the need for an explicit stack.

7 The stackless bvh traverse algorithm

Algorithm 16 Stackless Traversal Core Logic (Simplified)

```

1: procedure STACKLESSTRAVERSE(nodes, queryAABB)
2:   nodeIdX ← 0                                     ▷ Root is at index 0
3:   while nodeIdX ≠ -1 do
4:     node ← nodes[nodeIdX]
5:     if node.bound ∩ queryAABB ≠ ∅ then             ▷ HIT
6:       if node.data = -1 then                       ▷ Internal node
7:         nodeIdX ← nodeIdX + 1                     ▷ Left child is next in preorder
8:       else                                         ▷ Leaf node
9:         REPORTCOLLISION(node.data)
10:        nodeIdX ← node.escape                       ▷ Done with leaf
11:      end if
12:    else                                         ▷ MISS
13:      nodeIdX ← node.escape                       ▷ Skip entire subtree
14:    end if
15:  end while
16: end procedure
  
```

Algorithm 17 Single-Pass Stackless BVH Traversal

```
1: procedure STACKLESSQUERY(result, counter, maxRes, nodes, queryIDs, queryAABBs,  $N$ )
2:   tid  $\leftarrow$  globalThreadId
3:   objIdx  $\leftarrow$  queryIDs[tid]
4:   queryAABB  $\leftarrow$  queryAABBs[objIdx]
5:
6:   shared buffer[BUFFER_SIZE], bufferCount  $\leftarrow$  0, activeCount
7:   nodeId  $\leftarrow$  0 ▷ Start at root
8:   done  $\leftarrow$  false
9:
10:  while true do
11:    SYNC_THREADS
12:    if not done then
13:      while nodeId  $\neq$  -1 do
14:        node  $\leftarrow$  nodes[nodeId]
15:        hit  $\leftarrow$  node.bound  $\cap$  queryAABB  $\neq \emptyset$ 
16:
17:        if hit then
18:          if node.data  $\neq$  -1 then ▷ Leaf node
19:            ▷ Report collision
20:            slot  $\leftarrow$  ATOMICADD(bufferCount, 1)
21:            if slot  $\geq$  BUFFER_SIZE then
22:              ATOMICSUB(bufferCount, 1)
23:              break ▷ Buffer full, flush first
24:            end if
25:            buffer[slot]  $\leftarrow$  (node.data, objIdx)
26:            nodeId  $\leftarrow$  node.escape ▷ Go to escape
27:          else ▷ Internal node
28:            nodeId  $\leftarrow$  nodeId + 1 ▷ Go to left child
29:          end if
30:        else ▷ Miss
31:          nodeId  $\leftarrow$  node.escape ▷ Skip subtree
32:        end if
33:      end while
34:      if nodeId = -1 then
35:        done  $\leftarrow$  true
36:        ATOMICSUB(activeCount, 1)
37:      end if
38:    end if
39:
40:    ▷ Flush buffer to global memory
41:    SYNC_THREADS
42:    FLUSHBUFFER(result, counter, buffer, bufferCount)
43:    SYNC_THREADS
44:    if activeCount = 0 or counter  $\geq$  maxRes then
45:      return
46:    end if
47:  end while
48: end procedure
```

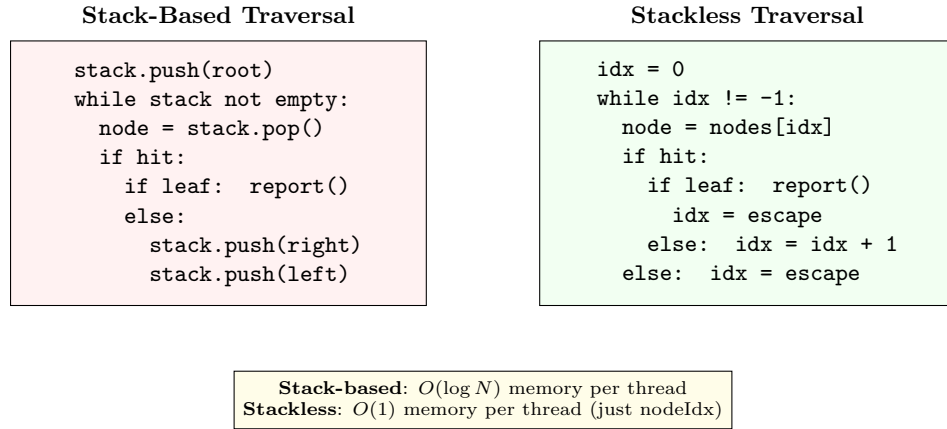


Figure 12: Comparison of stack-based vs stackless traversal. Stackless eliminates per-thread stack overhead.

Reference

- [1] Karras, Maximizing Parallelism in the Construction of BVHs, Octrees, and k-d Trees.
- [2] Damkj, Stackless BVH Collision Detection for Physical Simulation.
- [3] Wang et al., Efficient BVH-based Collision Detection Scheme with Ordering and Restructuring.
- [4] <https://github.com/ZiXuanVickyLu/culbvh>
- [5] <https://github.com/jerry060599/KittenGpuLBVH>
- [6] <https://min-tang.github.io/home/BVH-OR/>